

List Implementation #2: _____

Alternate List.h	
1	#pragma once
2	
3	template <typename T>
4	class List {
5	public:
...	/* ... */
28	private:
29	
30	
31	
32	};

Array - Implementation Details:

C	S	2	2	5
[0]	[1]	[2]	[3]	[4]

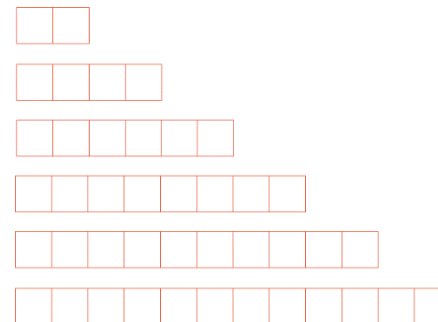
1. What is the running time of `[]`? How?
2. What is the running time of `insertFront()`? Why?
3. What is the running time of `insert()`? Why?

4. What is the running time of `remove()`? Why?

Implementation Details and Analysis:

➔ What is our resize strategy?

Array Resize Strategy #1:



...total copies across all resizes: _____

...total number of insert operations: _____

...average (amortized) cost of copies per insert: _____

What is our Big O runtime and amortized runtime?

Array Resize Strategy #2:



...total copies across all resizes: _____

...total number of insert operations: _____

...average (amortized) cost of copies per insert: _____

What is our Big O runtime and amortized runtime?

Running Time:

	Singly Linked List	Array
Look up arbitrary location		
Insert after a given element		
Remove after a given element		
Insert at arbitrary location		
Remove at arbitrary location		
Search for an input value		

Consider tradeoffs between data structures when deciding what to use! Can you think of some ways to improve some of the data structures seen today? What are the consequences?